

Система управления проектами

Базы данных



**Меня хорошо видно
& слышно?**



Защита проекта

Тема: Система управления проектами

Владимир Гришанин

Старший разработчик C#/.NET



План защиты

Цель и задачи проекта

Какие технологии использовались

Что получилось

Выводы

Вопросы и рекомендации

Цель и задачи проекта

Цель проекта: создать схему базы данных для системы управления проектами

1. Разработать схему базы данных.
2. Разработать методы модификации данных.
3. Определить типовые запросы к базе данных.
4. Добавить демо данные.

Этапы проектирования

Этап 1

Проектирование схемы базы данных:

1. Определить таблицы.
2. Выбрать колонки.
3. Спроектировать связи таблиц.

Этап 2

Проектирование методов модификации данных. Хранимые процедуры добавления, обновления и удаления данных.

Этап 3

Проектирование типовых запросов к базе данных. Их оптимизация. Проектирование индексов на таблицах.

Этап 4

Заполнение таблицы демонстрационными данными.

Какие технологии использовались

PostgreSQL

Docker

VS Code

DBeaver



Схема базы данных

Таблица Projects

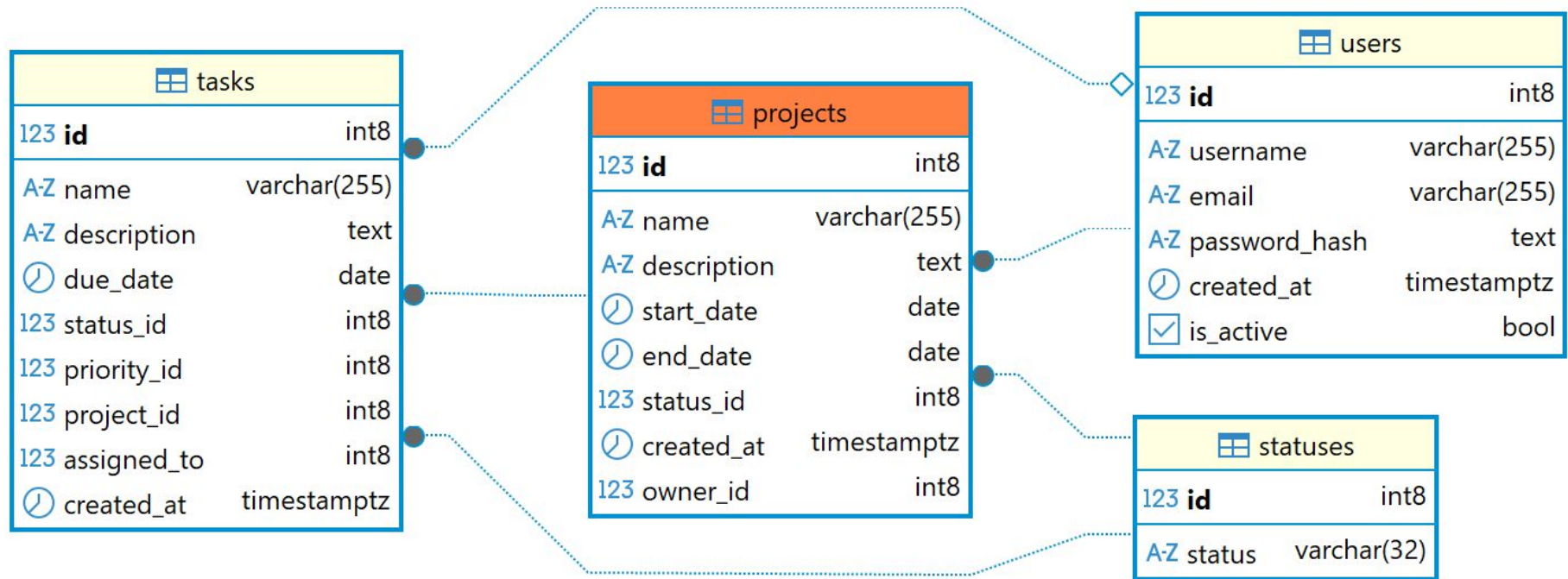


Схема базы данных

Таблица Tasks

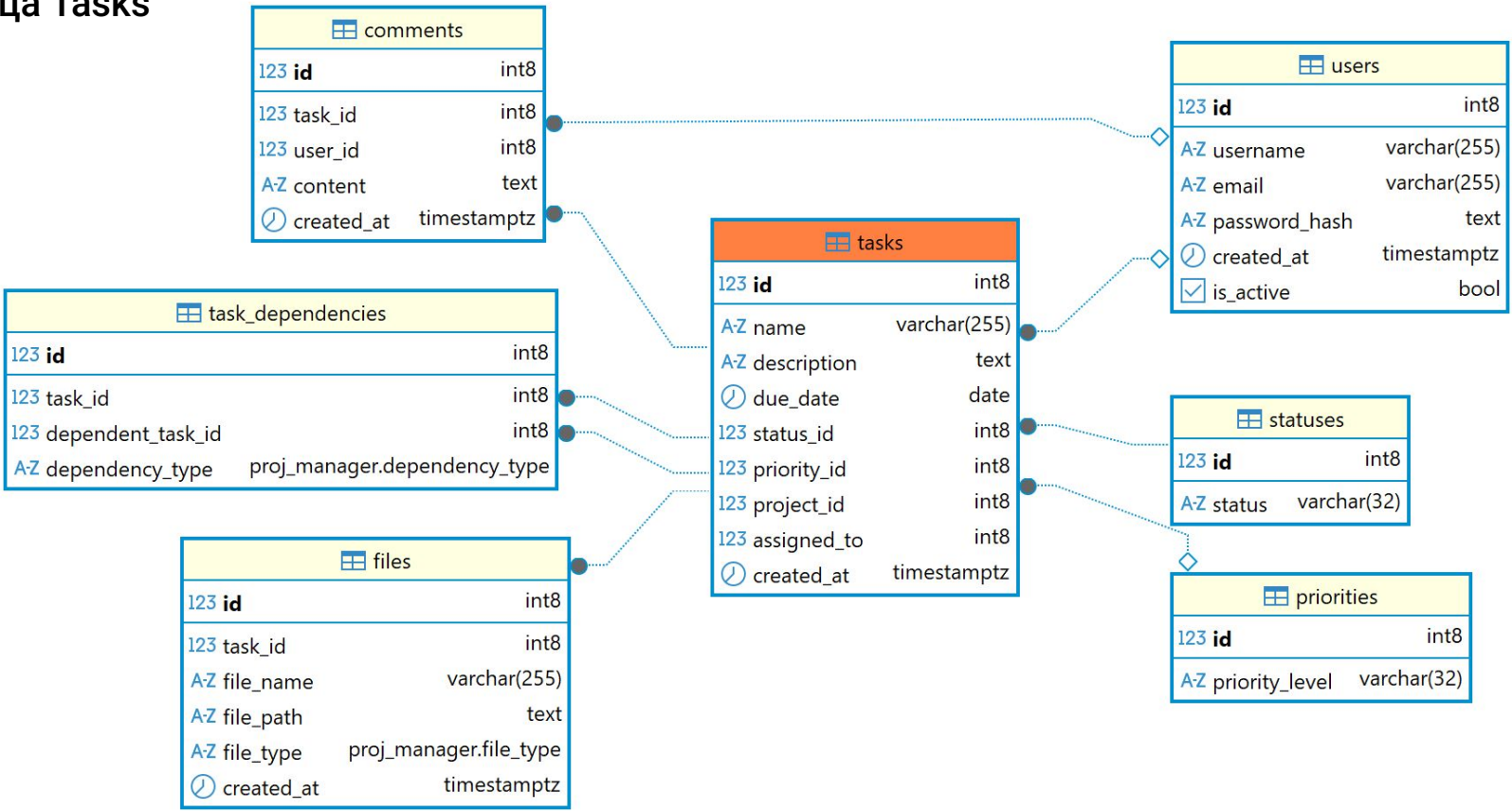


Схема базы данных

Таблица Teams

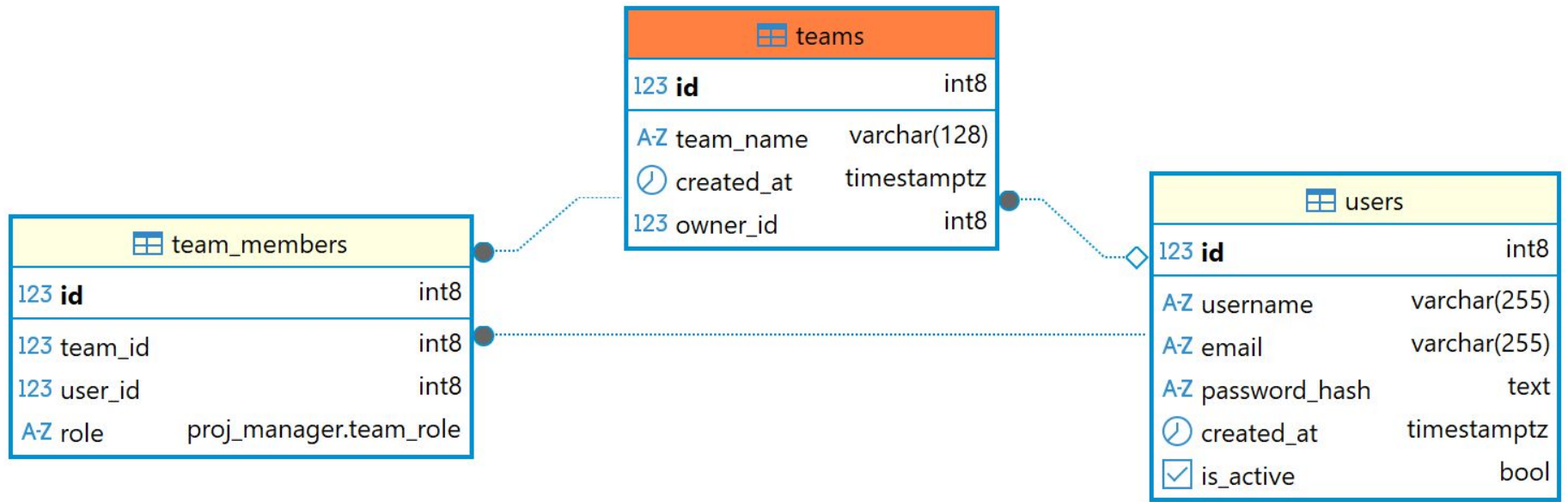


Схема базы данных

Таблица Time tracking

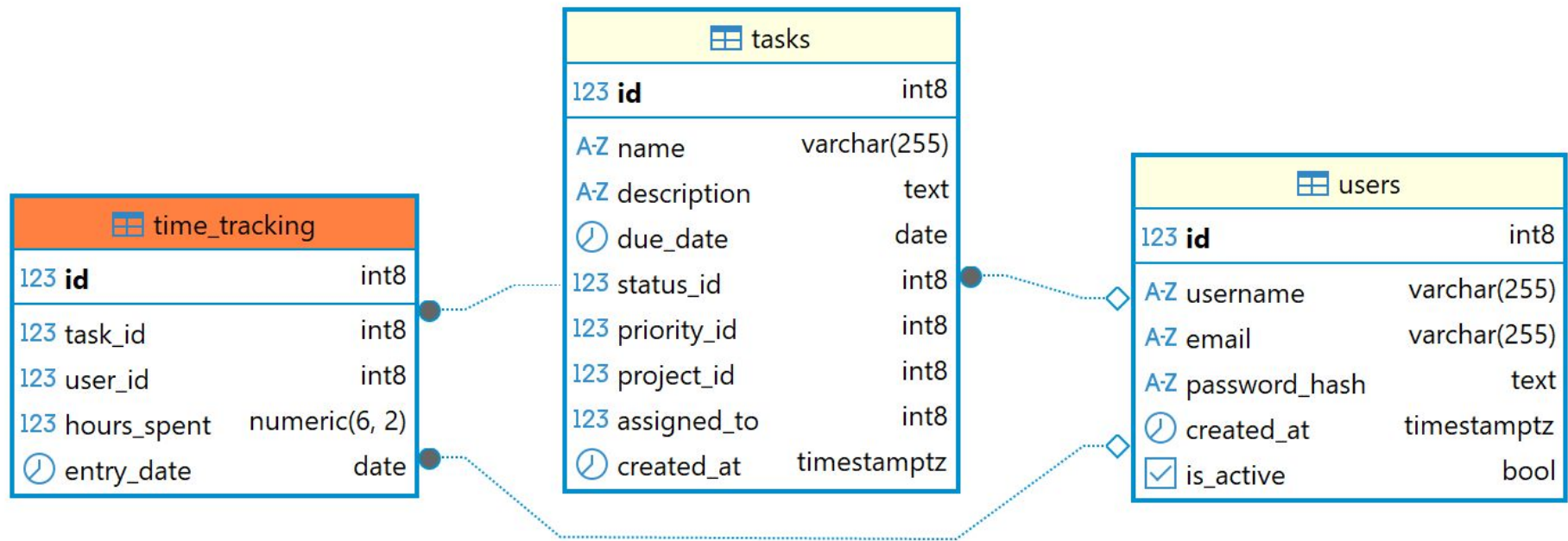
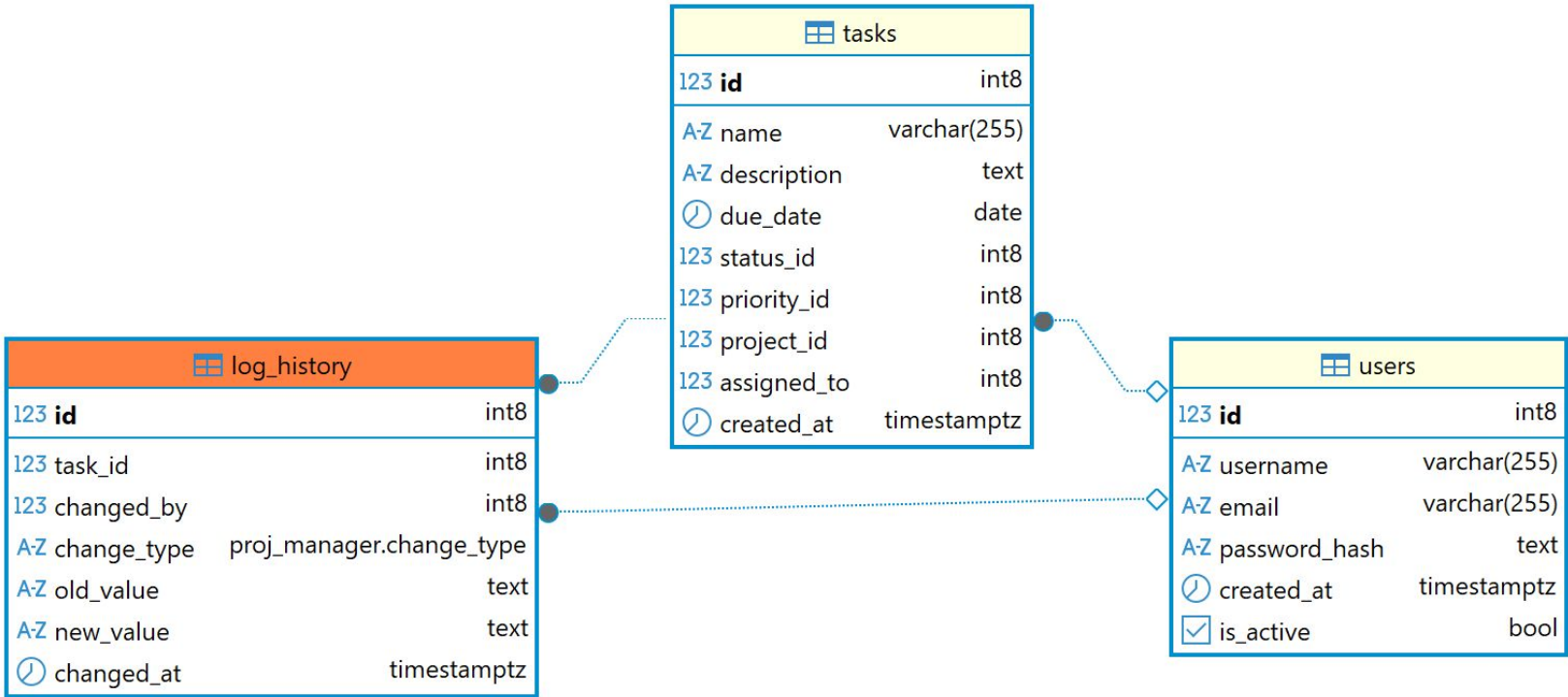


Схема базы данных

Таблица Log history



Хранимые процедуры

Таблица Projects ([stored_procedures.sql](#))

```
CREATE OR REPLACE PROCEDURE create_project(  
    p_name VARCHAR(255),  
    p_description TEXT,  
    p_start_date DATE,  
    p_end_date DATE,  
    p_status VARCHAR(32),  
    p_owner_name VARCHAR(255),  
    OUT p_error_code INT  
)
```

```
CREATE OR REPLACE PROCEDURE create_task(  
    p_name VARCHAR(255),  
    p_description TEXT,  
    p_due_date DATE,  
    p_status VARCHAR(32),  
    p_priority VARCHAR(32),  
    p_project_id BIGINT,  
    OUT p_error_code INT,  
    p_user_name VARCHAR(255) DEFAULT NULL  
)
```



Хранимые процедуры

Таблица Tasks ([stored_procedures.sql](#))

```
CREATE OR REPLACE PROCEDURE update_project(  
    p_project_id BIGINT,  
    OUT p_error_code INT,  
    p_name VARCHAR(255) DEFAULT NULL,  
    p_description TEXT DEFAULT NULL,  
    p_start_date DATE DEFAULT NULL,  
    p_end_date DATE DEFAULT NULL,  
    p_status VARCHAR(32) DEFAULT NULL  
)
```

```
CREATE OR REPLACE PROCEDURE update_task_with_log(  
    p_task_id BIGINT,  
    p_changed_by VARCHAR(255),  
    OUT p_error_code INT,  
    p_name VARCHAR(255) DEFAULT NULL,  
    p_description TEXT DEFAULT NULL,  
    p_due_date DATE DEFAULT NULL,  
    p_status VARCHAR(32) DEFAULT NULL,  
    p_priority VARCHAR(32) DEFAULT NULL,  
    p_assigned_to VARCHAR(255) DEFAULT NULL  
)
```

Хранимые процедуры

Таблица Tasks ([stored_procedures.sql](#))

```
CREATE OR REPLACE PROCEDURE add_comment_to_task(  
    p_task_id BIGINT,  
    p_user_name VARCHAR(128),  
    p_comment_content TEXT,  
    OUT p_error_code INT  
)
```

```
CREATE OR REPLACE PROCEDURE add_file_to_task(  
    p_task_id BIGINT,  
    p_file_name VARCHAR(255),  
    p_file_path TEXT,  
    p_file_type VARCHAR(32),  
    OUT p_error_code INT  
)
```



Хранимые процедуры

Таблица Teams ([stored_procedures.sql](#))

```
CREATE OR REPLACE PROCEDURE create_team(  
    p_team_name VARCHAR(128),  
    p_owner_name VARCHAR(128),  
    OUT p_error_code INT  
)
```

```
CREATE OR REPLACE PROCEDURE add_team_member(  
    p_team_name VARCHAR(128),  
    p_user_name VARCHAR(128),  
    p_role VARCHAR(32) DEFAULT 'member',  
    OUT p_error_code INT  
)
```



Запросы к БД

Выбор всех проектов с их статусами и именами владельцев

```
SELECT p.id,  
       p.name,  
       p.description,  
       p.start_date,  
       p.end_date,  
       u.username AS owner_username,  
       s.status AS status_name  
FROM projects p  
JOIN users u ON p.owner_id = u.id  
JOIN statuses s ON p.status_id = s.id;
```

Выбор всех задач для конкретного проекта с их статусами, приоритетами и именами назначенных пользователей

```
SELECT t.id,  
       t.name,  
       t.description,  
       t.due_date,  
       st.status AS status_name,  
       pr.priority_level,  
       u.username AS assigned_to_username  
FROM tasks t  
JOIN statuses st ON t.status_id = st.id  
JOIN priorities pr ON t.priority_id = pr.id  
JOIN users u ON t.assigned_to = u.id  
WHERE t.project_id = :project_id;
```



Запросы к БД

Выбор всех комментариев для конкретной задачи с именами комментаторов и датой создания

```
SELECT c.id,  
       c.content,  
       u.username AS commenter_username,  
       c.created_at  
FROM comments c  
JOIN users u ON c.user_id = u.id  
WHERE c.task_id = :task_id;
```

Выбор истории изменений для конкретной задачи с типами изменений, старыми и новыми значениями, датой изменения и именами пользователей, которые внесли изменения

```
SELECT lh.id,  
       lh.change_type,  
       lh.old_value,  
       lh.new_value,  
       lh.changed_at,  
       u.username AS changed_by_username  
FROM log_history lh  
JOIN users u ON lh.changed_by = u.id  
WHERE lh.task_id = :task_id;
```



Запросы к БД

Выбор членов команды конкретного проекта с их ролями

```
SELECT p.id AS project_id,  
       p.name AS project_name,  
       t.id AS team_id,  
       t.team_name,  
       u.id AS user_id,  
       u.username,  
       tm.role  
FROM projects p  
JOIN teams t ON p.owner_id = t.owner_id  
JOIN team_members tm ON t.id = tm.team_id  
JOIN users u ON tm.user_id = u.id  
WHERE p.id = :project_id;
```

Выбор времени, затраченного пользователями на конкретную задачу, с датой и именами пользователей

```
SELECT t.id,  
       t.name,  
       u.id AS user_id,  
       u.username,  
       tt.entry_date,  
       SUM(tt.hours_spent) AS total_hours  
FROM tasks t  
JOIN time_tracking tt ON t.id = tt.task_id  
LEFT JOIN users u ON tt.user_id = u.id  
WHERE tt.task_id = :task_id  
GROUP BY t.id, u.id, tt.entry_date  
ORDER BY tt.entry_date DESC;
```



Индексы

Таблица Tasks ([create_indexes.sql](#))

1. GIN индексы для полнотекстового поиска по имени и описанию задачи:
 - a. **idx_tasks_task_name_gin**
 - b. **idx_tasks_task_description_gin**
2. Составной индекс для поиска задач по проекту и сортировки по статусу
idx_tasks_project_id_status

```
CREATE INDEX idx_tasks_task_name_gin ON tasks USING gin (to_tsvector(name));  
CREATE INDEX idx_tasks_task_description_gin ON tasks USING gin (to_tsvector(description));  
CREATE INDEX idx_tasks_project_id_status ON tasks(project_id, status_id);
```

Индексы

Таблица Time tracking ([create_indexes.sql](#))

1. Индекс для поиска записей по task_id и user_id с включением полей hours_spent и entry_date:
idx_time_tracking_task_id_user_id
2. Составной индекс для поиска записей по task_id и entry_date с включением hours_spent
idx_time_tracking_task_id_entry_date

```
CREATE INDEX idx_time_tracking_task_id_user_id ON time_tracking(task_id, user_id) INCLUDE  
(hours_spent, entry_date);
```

```
CREATE INDEX idx_time_tracking_task_id_entry_date ON time_tracking(task_id, entry_date) INCLUDE  
(hours_spent);
```



Индексы

Прочие индексы:

1. Уникальный индекс для быстрого поиска по имени пользователя:
idx_users_username
2. Поиск истории изменений по task_id:
idx_log_history_task_id
3. Поиск комментариев по task_id и сортировка по created_at:
idx_comments_task_id_created_at

```
CREATE INDEX idx_users_username ON users(username);
```

```
CREATE INDEX idx_log_history_task_id ON log_history(task_id);
```

```
CREATE INDEX idx_comments_task_id_created_at ON comments(task_id, created_at);
```



Демо данные

Два проекта:

1. “Разработка фронтенд-приложения”.

Задачи:

- 1.1. “Разработка UI компонентов”
- 1.2. “Реализация UI аутентификации”

2. “Разработка бэкенд-сервисов для API”

Задачи:

- 2.1. “Настройка backend API”
- 2.2. “Дизайн базы данных”
- 2.3. “Документация API endpoint”



Демо данные

Две команды:

1. “Frontend Team”, Bob - lead. Члены команды:
 - 1.1. Frank - developer
 - 1.2. Grace - developer
 - 1.3. Heidi - tester
2. “Backend Team”, Carol - lead. Члены команды:
 - 2.1. Ivan - developer
 - 2.2. John - developer
 - 2.3. Max - devops



Демо

Выводы

1. Схема БД системы управления проектами спроектирована и реализована.
2. Созданы хранимые процедуры создания и обновления данных.
3. Разработаны основные запросы к БД.
4. Добавлены данные и продемонстрирована работа с ними.
5. Польза проекта для меня как для разработчика - 10/10.
6. Вопрос: реализовывать бизнес логику в хранимых процедурах или в приложении?

Вопросы и рекомендации



если есть вопросы



если вопросов нет



Спасибо за внимание!